

Segundos pasos en Python

Pensamiento Computacional

Facultad de Ciencias Exactas y Naturales, UBA

Segundo Cuatrimestre 2026

ADMINISTRATIVO

- **Campus de la materia:**

- **Campus de la materia:**

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.

● Campus de la materia:

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.
- Algunos tienen/tuvieron que terminar de crear una cuenta **IdEx** (recibieron un mail por esto, verifiquen en SPAM).

● Campus de la materia:

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.
- Algunos tienen/tuvieron que terminar de crear una cuenta **IdEx** (recibieron un mail por esto, verifiquen en SPAM).
- Algunos ya tenían usuario por haber hecho algún curso o tutoría de Exactas, esos pueden entrar directo.

● Campus de la materia:

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.
- Algunos tienen/tuvieron que terminar de crear una cuenta **IdEx** (recibieron un mail por esto, verifiquen en SPAM).
- Algunos ya tenían usuario por haber hecho algún curso o tutoría de Exactas, esos pueden entrar directo.
- Si alguien no dio el presente o tuvo algún problema con la matriculación: mail a `penscomp-docentes@dc.uba.ar` y/o `emocskos@dc.uba.ar` para avisar.

● Campus de la materia:

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.
- Algunos tienen/tuvieron que terminar de crear una cuenta **IdEx** (recibieron un mail por esto, verifiquen en SPAM).
- Algunos ya tenían usuario por haber hecho algún curso o tutoría de Exactas, esos pueden entrar directo.
- Si alguien no dio el presente o tuvo algún problema con la matriculación: mail a `penscomp-docentes@dc.uba.ar` y/o `emocskos@dc.uba.ar` para avisar.
- Hay un **cuestionario** de repaso de la clase anterior (muy pocos lo completaron). Vamos a ir dejando tareas en el campus, si quieren aprobar el curso, les conviene hacerlas, cada una no lleva mucho tiempo y sirve para repasar lo que vimos.

● Campus de la materia:

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.
- Algunos tienen/tuvieron que terminar de crear una cuenta **IdEx** (recibieron un mail por esto, verifiquen en SPAM).
- Algunos ya tenían usuario por haber hecho algún curso o tutoría de Exactas, esos pueden entrar directo.
- Si alguien no dio el presente o tuvo algún problema con la matriculación: mail a `penscomp-docentes@dc.uba.ar` y/o `emocskos@dc.uba.ar` para avisar.
- Hay un **cuestionario** de repaso de la clase anterior (muy pocos lo completaron). Vamos a ir dejando tareas en el campus, si quieren aprobar el curso, les conviene hacerlas, cada una no lleva mucho tiempo y sirve para repasar lo que vimos.

● Fechas de Parcial:

● Campus de la materia:

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.
- Algunos tienen/tuvieron que terminar de crear una cuenta **IdEx** (recibieron un mail por esto, verifiquen en SPAM).
- Algunos ya tenían usuario por haber hecho algún curso o tutoría de Exactas, esos pueden entrar directo.
- Si alguien no dio el presente o tuvo algún problema con la matriculación: mail a `penscomp-docentes@dc.uba.ar` y/o `emocskos@dc.uba.ar` para avisar.
- Hay un **cuestionario** de repaso de la clase anterior (muy pocos lo completaron). Vamos a ir dejando tareas en el campus, si quieren aprobar el curso, les conviene hacerlas, cada una no lleva mucho tiempo y sirve para repasar lo que vimos.

● Fechas de Parcial:

- **Primer parcial:** sábado 26 de septiembre (de 13 a 17).

● Campus de la materia:

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.
- Algunos tienen/tuvieron que terminar de crear una cuenta **IdEx** (recibieron un mail por esto, verifiquen en SPAM).
- Algunos ya tenían usuario por haber hecho algún curso o tutoría de Exactas, esos pueden entrar directo.
- Si alguien no dio el presente o tuvo algún problema con la matriculación: mail a `penscomp-docentes@dc.uba.ar` y/o `emocskos@dc.uba.ar` para avisar.
- Hay un **cuestionario** de repaso de la clase anterior (muy pocos lo completaron). Vamos a ir dejando tareas en el campus, si quieren aprobar el curso, les conviene hacerlas, cada una no lleva mucho tiempo y sirve para repasar lo que vimos.

● Fechas de Parcial:

- **Primer parcial:** sábado 26 de septiembre (de 13 a 17).
- **Segundo parcial:** sábado 31 de octubre (de 13 a 17)

● Campus de la materia:

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.
- Algunos tienen/tuvieron que terminar de crear una cuenta **IdEx** (recibieron un mail por esto, verifiquen en SPAM).
- Algunos ya tenían usuario por haber hecho algún curso o tutoría de Exactas, esos pueden entrar directo.
- Si alguien no dio el presente o tuvo algún problema con la matriculación: mail a `penscomp-docentes@dc.uba.ar` y/o `emocskos@dc.uba.ar` para avisar.
- Hay un **cuestionario** de repaso de la clase anterior (muy pocos lo completaron). Vamos a ir dejando tareas en el campus, si quieren aprobar el curso, les conviene hacerlas, cada una no lleva mucho tiempo y sirve para repasar lo que vimos.

● Fechas de Parcial:

- **Primer parcial:** sábado 26 de septiembre (de 13 a 17).
- **Segundo parcial:** sábado 31 de octubre (de 13 a 17)
- **Recuperatorio:** sábado 14 de noviembre (de 13 a 17)

● Campus de la materia:

- Todos los que dieron el presente **ya** están matriculados en el Campus de la materia.
- Algunos tienen/tuvieron que terminar de crear una cuenta **IdEx** (recibieron un mail por esto, verifiquen en SPAM).
- Algunos ya tenían usuario por haber hecho algún curso o tutoría de Exactas, esos pueden entrar directo.
- Si alguien no dio el presente o tuvo algún problema con la matriculación: mail a `penscomp-docentes@dc.uba.ar` y/o `emocskos@dc.uba.ar` para avisar.
- Hay un **cuestionario** de repaso de la clase anterior (muy pocos lo completaron). Vamos a ir dejando tareas en el campus, si quieren aprobar el curso, les conviene hacerlas, cada una no lleva mucho tiempo y sirve para repasar lo que vimos.

● Fechas de Parcial:

- **Primer parcial:** sábado 26 de septiembre (de 13 a 17).
- **Segundo parcial:** sábado 31 de octubre (de 13 a 17)
- **Recuperatorio:** sábado 14 de noviembre (de 13 a 17)

- En el Campus hay **ejercitación**. Es para ustedes, no para nosotros ;-).

RECORDANDO LO ANTERIOR

¿En qué estábamos?



El problemas

- Dado un **álbum** con 980 casilleros.

El problemas

- Dado un **álbum** con 980 casilleros.
- Compramos figuritas numeradas, para **pegar** en el álbum con el objetivo de llenarlo.

El problemas

- Dado un **álbum** con 980 casilleros.
- Compramos figuritas numeradas, para **pegar** en el álbum con el objetivo de llenarlo.
- Se supone que las figuritas se imprimen y distribuyen de manera totalmente **aleatoria**.

El problemas

- Dado un **álbum** con 980 casilleros.
- Comparamos figuritas numeradas, para **pegar** en el álbum con el objetivo de llenarlo.
- Se supone que las figuritas se imprimen y distribuyen de manera totalmente **aleatoria**.
- **Queremos responder:**

El problemas

- Dado un **álbum** con 980 casilleros.
- Compramos figuritas numeradas, para **pegar** en el álbum con el objetivo de llenarlo.
- Se supone que las figuritas se imprimen y distribuyen de manera totalmente **aleatoria**.
- **Queremos responder:**
 - ¿Cuántas figuritas hay que comprar **en promedio** para llenar el álbum?

El problemas

- Dado un **álbum** con 980 casilleros.
- Compramos figuritas numeradas, para **pegar** en el álbum con el objetivo de llenarlo.
- Se supone que las figuritas se imprimen y distribuyen de manera totalmente **aleatoria**.
- **Queremos responder:**
 - ¿Cuántas figuritas hay que comprar **en promedio** para llenar el álbum?
 - Si tengo plata para comprar 1000 figuritas, ¿qué **chances** tengo de llenarlo?

Modelado

- Cada figurita es un **número** (sólo nos importa en qué casillero del álbum se pega). Todo el resto de los detalles de la vida real, los descartamos.

Modelado

- Cada figurita es un **número** (sólo nos importa en qué casillero del álbum se pega). Todo el resto de los detalles de la vida real, los descartamos.
- Para simular la aleatoriedad al comprar una figurita, usamos el módulo `random`. Agregamos la instrucción `import random` al principio de nuestro código.

Modelado

- Cada figurita es un **número** (sólo nos importa en qué casillero del álbum se pega). Todo el resto de los detalles de la vida real, los descartamos.
- Para simular la aleatoriedad al comprar una figurita, usamos el módulo `random`. Agregamos la instrucción `import random` al principio de nuestro código.
- El álbum se modela como una **lista**. Cada casillero para poner una figurita es un 0 en la lista, al pegarla, lo cambiamos por un 1.

```
import random
```

```
figu = random.randint(0,5)
```

```
import random  
  
figu = random.randint(0,5)
```

- **import random**: importa el **módulo** `random`, poniendo sus *funciones* a disposición.

```
import random  
  
figu = random.randint(0,5)
```

- `import random`: importa el **módulo** `random`, poniendo sus *funciones* a disposición.
- `random.randint(0,5)`:

```
import random  
  
figu = random.randint(0,5)
```

- **import random**: importa el **módulo** `random`, poniendo sus *funciones* a disposición.
- `random.randint(0,5)`:
 - `randint(a,b)` es una función del módulo `random`, por eso se usa `random` como prefijo.

```
import random  
  
figu = random.randint(0,5)
```

- **import random**: importa el **módulo** `random`, poniendo sus *funciones* a disposición.
- `random.randint(0,5)`:
 - `randint(a,b)` es una función del módulo `random`, por eso se usa `random` como prefijo.
 - `randint(a,b)`: recibe dos números enteros `a` y `b`, y devuelve un número entero elegido aleatoriamente entre `a` y `b` (se incluyen ambos extremos).

```
import random  
  
figu = random.randint(0,5)
```

- **import random**: importa el **módulo** `random`, poniendo sus *funciones* a disposición.
- `random.randint(0,5)`:
 - `randint(a,b)` es una función del módulo `random`, por eso se usa `random` como prefijo.
 - `randint(a,b)`: recibe dos números enteros `a` y `b`, y devuelve un número entero elegido aleatoriamente entre `a` y `b` (se incluyen ambos extremos).
- `figu = random.randint(0,5)` **define** una variable:

```
import random  
  
figu = random.randint(0,5)
```

- **import random**: importa el **módulo** `random`, poniendo sus *funciones* a disposición.
- `random.randint(0,5)`:
 - `randint(a,b)` es una función del módulo `random`, por eso se usa `random` como prefijo.
 - `randint(a,b)`: recibe dos números enteros `a` y `b`, y devuelve un número entero elegido aleatoriamente entre `a` y `b` (se incluyen ambos extremos).
- `figu = random.randint(0,5)` **define** una variable:
 - Evalúa la función `randint` con los parámetros de entrada `0` y `5`. Obtiene como resultado un número aleatorio entre `0` y `5`.

```
import random  
  
figu = random.randint(0,5)
```

- **import random**: importa el **módulo** `random`, poniendo sus *funciones* a disposición.
- `random.randint(0,5)`:
 - `randint(a,b)` es una función del módulo `random`, por eso se usa `random` como prefijo.
 - `randint(a,b)`: recibe dos números enteros `a` y `b`, y devuelve un número entero elegido aleatoriamente entre `a` y `b` (se incluyen ambos extremos).
- `figu = random.randint(0,5)` **define** una variable:
 - Evalúa la función `randint` con los parámetros de entrada `0` y `5`. Obtiene como resultado un número aleatorio entre `0` y `5`.
 - Una vez obtenido el valor, se lo almacena en memoria y se lo etiqueta con el nombre `figu`.

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del = (sea lo que sea).

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del = (sea lo que sea).
- Reserva un espacio en **memoria**.

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del = (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del = (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.
- Etiqueta ese **espacio** con el nombre de la derecha (en este caso, `figu`), que es el nombre dado a la variable.

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del = (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.
- Etiqueta ese **espacio** con el nombre de la derecha (en este caso, `figu`), que es el nombre dado a la variable.
- El nombre lo elegimos nosotros, se busca siempre algo que nos dé una **pista** de para qué vamos a usar esa variable.

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del `=` (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.
- Etiqueta ese **espacio** con el nombre de la derecha (en este caso, `figu`), que es el nombre dado a la variable.
- El nombre lo elegimos nosotros, se busca siempre algo que nos dé una **pista** de para qué vamos a usar esa variable.
- `Python` nos permite usar **casi** cualquier cosa como nombre de variable. Está **prohibido** usar:

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del = (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.
- Etiqueta ese **espacio** con el nombre de la derecha (en este caso, `figu`), que es el nombre dado a la variable.
- El nombre lo elegimos nosotros, se busca siempre algo que nos dé una **pista** de para qué vamos a usar esa variable.
- **Python** nos permite usar **casi** cualquier cosa como nombre de variable. Está **prohibido** usar:
 - Números, o expresiones que *comiencen* con números (ejemplo: `123pp`).

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del = (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.
- Etiqueta ese **espacio** con el nombre de la derecha (en este caso, `figu`), que es el nombre dado a la variable.
- El nombre lo elegimos nosotros, se busca siempre algo que nos dé una **pista** de para qué vamos a usar esa variable.
- **Python** nos permite usar **casi** cualquier cosa como nombre de variable. Está **prohibido** usar:
 - Números, o expresiones que *comiencen* con números (ejemplo: `123pp`).
 - Palabras clave que usa **Python** para funcionar, como `while` (veremos otras).

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del `=` (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.
- Etiqueta ese **espacio** con el nombre de la derecha (en este caso, `figu`), que es el nombre dado a la variable.
- El nombre lo elegimos nosotros, se busca siempre algo que nos dé una **pista** de para qué vamos a usar esa variable.
- **Python** nos permite usar **casi** cualquier cosa como nombre de variable. Está **prohibido** usar:
 - Números, o expresiones que *comiencen* con números (ejemplo: `123pp`).
 - Palabras clave que usa **Python** para funcionar, como `while` (veremos otras).
 - Expresiones con espacios.

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del `=` (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.
- Etiqueta ese **espacio** con el nombre de la derecha (en este caso, `figu`), que es el nombre dado a la variable.
- El nombre lo elegimos nosotros, se busca siempre algo que nos dé una **pista** de para qué vamos a usar esa variable.
- **Python** nos permite usar **casi** cualquier cosa como nombre de variable. Está **prohibido** usar:
 - Números, o expresiones que *comiencen* con números (ejemplo: `123pp`).
 - Palabras clave que usa **Python** para funcionar, como `while` (veremos otras).
 - Expresiones con espacios.
- Conviene poner algo que tenga **sentido** para un humano, para que el código sea más fácil de entender.

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del `=` (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.
- Etiqueta ese **espacio** con el nombre de la derecha (en este caso, `figu`), que es el nombre dado a la variable.
- El nombre lo elegimos nosotros, se busca siempre algo que nos dé una **pista** de para qué vamos a usar esa variable.
- **Python** nos permite usar **casi** cualquier cosa como nombre de variable. Está **prohibido** usar:
 - Números, o expresiones que *comiencen* con números (ejemplo: `123pp`).
 - Palabras clave que usa **Python** para funcionar, como `while` (veremos otras).
 - Expresiones con espacios.
- Conviene poner algo que tenga **sentido** para un humano, para que el código sea más fácil de entender.
- El nombre nos permite **recuperar** el valor almacenado, para volver a usarlo.

Asignación, detalles

La instrucción

```
figu = random.randint(0,5)
```

es una **asignación**.

- **Primero**, evalúa la operación ubicada a la derecha del `=` (sea lo que sea).
- Reserva un espacio en **memoria**.
- **Almacena** el resultado de la operación de la derecha en ese espacio en memoria.
- Etiqueta ese **espacio** con el nombre de la derecha (en este caso, `figu`), que es el nombre dado a la variable.
- El nombre lo elegimos nosotros, se busca siempre algo que nos dé una **pista** de para qué vamos a usar esa variable.
- **Python** nos permite usar **casi** cualquier cosa como nombre de variable. Está **prohibido** usar:
 - Números, o expresiones que *comiencen* con números (ejemplo: `123pp`).
 - Palabras clave que usa **Python** para funcionar, como `while` (veremos otras).
 - Expresiones con espacios.
- Conviene poner algo que tenga **sentido** para un humano, para que el código sea más fácil de entender.
- El nombre nos permite **recuperar** el valor almacenado, para volver a usarlo.
- Se puede *sobreescribir* un nombre existente. En este caso, se **pierde** el valor **anterior**.

Listas

Una **lista** es un *contenedor* que nos permite almacenar varios elementos bajo un mismo nombre:

```
album = [0,0,0,0,0,0]
```

Aquí creamos una lista de 6 elementos, llena de ceros.

Listas

Una **lista** es un *contenedor* que nos permite almacenar varios elementos bajo un mismo nombre:

```
album = [0,0,0,0,0,0]
```

Aquí creamos una lista de 6 elementos, llena de ceros.

Modelo

Estamos usando el 0 para codificar un casillero *vacío* en el álbum. Y usaremos un 1 para indicar que el casillero tiene la figurita pegada. De este modo:

```
[0,0,0,0,1,0]
```

representa un álbum que sólo tiene pegada la anteúltima figurita.

Indexado

- `Python` numera desde 0.

Indexado

- Python numera desde 0.
- Es decir que el primer elemento de una lista tiene el *índice* 0 y el último elemento de la lista tiene *índice* igual a la longitud de la lista menos 1.

Indexado

- Python numera desde 0.
- Es decir que el primer elemento de una lista tiene el *índice* 0 y el último elemento de la lista tiene *índice* igual a la longitud de la lista menos 1.
- `album = [0,0,0,0,0,0]`

Indexado

- Python numera desde 0.
- Es decir que el primer elemento de una lista tiene el *índice* 0 y el último elemento de la lista tiene *índice* igual a la longitud de la lista menos 1.
- `album = [0,0,0,0,0,0]`
- `len(album)` # devuelve 6

Indexado

- Python numera desde 0.
- Es decir que el primer elemento de una lista tiene el *índice* 0 y el último elemento de la lista tiene *índice* igual a la longitud de la lista menos 1.
- `album = [0,0,0,0,0,0]`
- `len(album)` # devuelve 6
- `album[0] = 1` # modifica el primer casillero y pone un 1

Indexado

- Python numera desde 0.
- Es decir que el primer elemento de una lista tiene el *índice* 0 y el último elemento de la lista tiene *índice* igual a la longitud de la lista menos 1.
- `album = [0,0,0,0,0,0]`
- `len(album)` # devuelve 6
- `album[0] = 1` # modifica el primer casillero y pone un 1
- `album[5] = 1` # modifica el \ 'ultimo casillero y pone un 1

Indexado

- Python numera desde 0.
- Es decir que el primer elemento de una lista tiene el *índice* 0 y el último elemento de la lista tiene *índice* igual a la longitud de la lista menos 1.
- `album = [0,0,0,0,0,0]`
- `len(album)` # devuelve 6
- `album[0] = 1` # modifica el primer casillero y pone un 1
- `album[5] = 1` # modifica el \ 'ultimo casillero y pone un 1
- Si se corren estas lineas se tiene que `album` es la lista: `[1,0,0,0,0,1]`.

Indexado

- Python numera desde 0.
- Es decir que el primer elemento de una lista tiene el *índice* 0 y el último elemento de la lista tiene *índice* igual a la longitud de la lista menos 1.
- `album = [0,0,0,0,0,0]`
- `len(album)` # devuelve 6
- `album[0] = 1` # modifica el primer casillero y pone un 1
- `album[5] = 1` # modifica el último casillero y pone un 1
- Si se corren estas líneas se tiene que `album` es la lista: `[1,0,0,0,0,1]`.
- `album[6] = 1` # DA ERROR! Fuera de rango

Esquema

Con estos elementos nuestro primer programa luciría así:

```
import random

album = [0,0,0,0,0,0]
Mientras el album no est\ 'e lleno...
    figu = random.randint(0,5)
    album[figu] = 1
```

Esquema

Con estos elementos nuestro primer programa luciría así:

```
import random

album = [0,0,0,0,0,0]
Mientras el album no est\ 'e lleno...
    figu = random.randint(0,5)
    album[figu] = 1
```

El código para decir *Mientras* es `while`:

Esquema

Con estos elementos nuestro primer programa luciría así:

```
import random

album = [0,0,0,0,0,0]
Mientras el album no est\ 'e lleno...
    figu = random.randint(0,5)
    album[figu] = 1
```

El código para decir *Mientras* es `while`:

```
import random

album = [0,0,0,0,0,0]
while [ALBUM NO LLENO]:
    figu = random.randint(0,5)
    album[figu] = 1
```

Tabulado

Python **sabe** qué parte del código tiene que repetir porque aparece *tabulado*, es decir: movido 4 espacios (o un `tab`) a la derecha.

Acá, se repiten las dos líneas siguientes al `while` *mientras se cumpla la condición*.

Modelo

Para representar la noción de que *el álbum no está lleno*, aprovechamos la manera en que modelamos el álbum:

Sumar los elementos de la lista nos indica la cantidad de **figuritas pegadas** (suman los unos y los ceros no aportan).

```
import random
album = [0,0,0,0,0,0]
while sum(album)<6:
    figu = random.randint(0,5)
    album[figu] = 1
```

Modelo

Para representar la noción de que *el álbum no está lleno*, aprovechamos la manera en que modelamos el álbum:

Sumar los elementos de la lista nos indica la cantidad de **figuritas pegadas** (suman los unos y los ceros no aportan).

```
import random
tam_album = 6
album = [0]*tam_album
while sum(album)<itam_album:
    figu = random.randint(0,tam_album-1)
    album[figu] = 1
```

Contar cuántas figus:

El código **funciona**, pero no nos da información interesante.

Contar cuántas figus:

El código **funciona**, pero no nos da información interesante.
Necesitamos registrar la cantidad de figuritas compradas

```
import random
tam_album = 6
cant_figus = 0
album = [0]*tam_album
while sum(album)<itam_album:
    figu = random.randint(0,itam_album-1)
    album[figu] = 1
    cant_figus = cant_figus + 1

print(cant_figus)
```

Contar cuántas figus:

El código **funciona**, pero no nos da información interesante.
Necesitamos registrar la cantidad de figuritas compradas

```
import random
tam_album = 6
cant_figus = 0
album = [0]*tam_album
while sum(album)<itam_album:
    figu = random.randint(0,itam_album-1)
    album[figu] = 1
    cant_figus = cant_figus + 1

print(cant_figus)
```

- `cant_figus` es una variable que **almacena** un entero.

Contar cuántas figus:

El código **funciona**, pero no nos da información interesante.
Necesitamos registrar la cantidad de figuritas compradas

```
import random
tam_album = 6
cant_figus = 0
album = [0]*tam_album
while sum(album)<itam_album:
    figu = random.randint(0,itam_album-1)
    album[figu] = 1
    cant_figus = cant_figus + 1

print(cant_figus)
```

- `cant_figus` es una variable que **almacena** un entero.
- Inicia en 0 porque al principio no compramos **ninguna** figu.

Contar cuántas figus:

El código **funciona**, pero no nos da información interesante.
Necesitamos registrar la cantidad de figuritas compradas

```
import random
tam_album = 6
cant_figus = 0
album = [0]*tam_album
while sum(album)<itam_album:
    figu = random.randint(0,itam_album-1)
    album[figu] = 1
    cant_figus = cant_figus + 1

print(cant_figus)
```

- `cant_figus` es una variable que **almacena** un entero.
- Inicia en 0 porque al principio no compramos **ninguna figu**.
- Lo aumentamos en una unidad en cada **iteración**, es decir: cada vez que compramos una figu.

Contar cuántas figus:

El código **funciona**, pero no nos da información interesante.
Necesitamos registrar la cantidad de figuritas compradas

```
import random
tam_album = 6
cant_figus = 0
album = [0]*tam_album
while sum(album)<itam_album:
    figu = random.randint(0,itam_album-1)
    album[figu] = 1
    cant_figus = cant_figus + 1

print(cant_figus)
```

- `cant_figus` es una variable que **almacena** un entero.
- Inicia en 0 porque al principio no compramos **ninguna figu**.
- Lo aumentamos en una unidad en cada **iteración**, es decir: cada vez que compramos una figu.
- **Imprimimos** el valor al final (fuera del while, antes de terminar el programa).

Cantidad promedio de figuritas

Hasta aquí, lo básico. Ahora queremos aprovechar este código para estimar la cantidad **promedio** de figus necesarias para llenar el álbum.

Queremos:

- Simular **muchas** veces el llenado de un álbum.

Cantidad promedio de figuritas

Hasta aquí, lo básico. Ahora queremos aprovechar este código para estimar la cantidad **promedio** de figus necesarias para llenar el álbum.

Queremos:

- Simular **muchas** veces el llenado de un álbum.
- **Registrar** cuántas figus hay que comprar cada vez.

Cantidad promedio de figuritas

Hasta aquí, lo básico. Ahora queremos aprovechar este código para estimar la cantidad **promedio** de figus necesarias para llenar el álbum.

Queremos:

- Simular **muchas** veces el llenado de un álbum.
- **Registrar** cuántas figus hay que comprar cada vez.
- Tomar el **promedio** todos los valores registrados.

Ejercicio

Supongamos que tenemos una lista, por ejemplo:

```
cantidades = [10,15,25,11,14,29,43,6,8,13]
```

Queremos escribir un código que calcule el promedio de los valores almacenados en la lista. Es decir: hay que sumarlos y dividir por la cantidad de elementos.

Ejercicio

Supongamos que tenemos una lista, por ejemplo:

```
cantidades = [10,15,25,11,14,29,43,6,8,13]
```

Queremos escribir un código que calcule el promedio de los valores almacenados en la lista. Es decir: hay que sumarlos y dividir por la cantidad de elementos.

- El ejemplo es **sólo un ejemplo**.
- El **mismo** código debería servir para promediar **otra** lista.
- Incluso debería servir si damos otra lista, de otra longitud.

Ejercicio

Supongamos que tenemos una lista, por ejemplo:

```
cantidades = [10,15,25,11,14,29,43,6,8,13]
```

Queremos escribir un código que calcule el promedio de los valores almacenados en la lista. Es decir: hay que sumarlos y dividir por la cantidad de elementos.

- El ejemplo es **sólo un ejemplo**.
- El **mismo** código debería servir para promediar **otra** lista.
- Incluso debería servir si damos otra lista, de otra longitud.

Versión fácil:

```
lista = [10,15,25,11,14,29,43,6,8,13]  
promedio = sum(lista)/len(lista)
```

Ejercicio

Supongamos que tenemos una lista, por ejemplo:

```
cantidades = [10,15,25,11,14,29,43,6,8,13]
```

Queremos escribir un código que calcule el promedio de los valores almacenados en la lista. Es decir: hay que sumarlos y dividir por la cantidad de elementos.

- El ejemplo es **sólo un ejemplo**.
- El **mismo** código debería servir para promediar **otra** lista.
- Incluso debería servir si damos otra lista, de otra longitud.

Versión fácil:

```
lista = [10,15,25,11,14,29,43,6,8,13]  
promedio = sum(lista)/len(lista)
```

¿Y si **no podemos** usar la función **sum**?

Ejercicio

Supongamos que tenemos una lista, por ejemplo:

```
cantidades = [10,15,25,11,14,29,43,6,8,13]
```

Queremos escribir un código que calcule el promedio de los valores almacenados en la lista. Es decir: hay que sumarlos y dividir por la cantidad de elementos.

- El ejemplo es **sólo un ejemplo**.
- El **mismo** código debería servir para promediar **otra** lista.
- Incluso debería servir si damos otra lista, de otra longitud.

Versión fácil:

```
lista = [10,15,25,11,14,29,43,6,8,13]  
promedio = sum(lista)/len(lista)
```

¿Y si **no podemos** usar la función **sum**?

No queda otra...

¡A trabajar!

Promedio

```
lista = [10,15,25,11,14,29,43,6,8,13]
suma = 0
i = 0
while i<len(lista):
    suma = suma + lista[i]
    i = i+1
promedio = suma/len(lista)
```

Promedio

```
lista = [10,15,25,11,14,29,43,6,8,13]
suma = 0
i = 0
while i<len(lista):
    suma = suma + lista[i]
    i = i+1
promedio = suma/len(lista)
```

- Para sumar los elementos necesitamos **recorrer** la lista, casillero a casillero.

Promedio

```
lista = [10,15,25,11,14,29,43,6,8,13]
suma = 0
i = 0
while i<len(lista):
    suma = suma + lista[i]
    i = i+1
promedio = suma/len(lista)
```

- Para sumar los elementos necesitamos **recorrer** la lista, casillero a casillero.
- Para recorrer la lista necesitamos definir una variable que recorra los **índices**.

Promedio

```
lista = [10,15,25,11,14,29,43,6,8,13]
suma = 0
i = 0
while i<len(lista):
    suma = suma + lista[i]
    i = i+1
promedio = suma/len(lista)
```

- Para sumar los elementos necesitamos **recorrer** la lista, casillero a casillero.
- Para recorrer la lista necesitamos definir una variable que recorra los **índices**.
- Una **convención** usual es llamar `i` (de *index*) a una variable que recorre índices.

Promedio

```
lista = [10,15,25,11,14,29,43,6,8,13]
suma = 0
i = 0
while i<len(lista):
    suma = suma + lista[i]
    i = i+1
promedio = suma/len(lista)
```

- Para sumar los elementos necesitamos **recorrer** la lista, casillero a casillero.
- Para recorrer la lista necesitamos definir una variable que recorra los **índices**.
- Una **convención** usual es llamar *i* (de *index*) a una variable que recorre índices.
- Necesitamos una variable que vaya registrando la suma **acumulando** la suma.

Promedio

```
lista = [10,15,25,11,14,29,43,6,8,13]
suma = 0
i = 0
while i<len(lista):
    suma = suma + lista[i]
    i = i+1
promedio = suma/len(lista)
```

- Para sumar los elementos necesitamos **recorrer** la lista, casillero a casillero.
- Para recorrer la lista necesitamos definir una variable que recorra los **índices**.
- Una **convención** usual es llamar `i` (de *index*) a una variable que recorre índices.
- Necesitamos una variable que vaya registrando la suma **acumulando** la suma.
- Cuando termina el `while`, la variable `suma` almacena la suma de **todos** los elementos de la lista.

Promedio

```
lista = [10,15,25,11,14,29,43,6,8,13]
suma = 0
i = 0
while i<len(lista):
    suma = suma + lista[i]
    i = i+1
promedio = suma/len(lista)
```

- Para sumar los elementos necesitamos **recorrer** la lista, casillero a casillero.
- Para recorrer la lista necesitamos definir una variable que recorra los **índices**.
- Una **convención** usual es llamar *i* (de *index*) a una variable que recorre índices.
- Necesitamos una variable que vaya registrando la suma **acumulando** la suma.
- Cuando termina el `while`, la variable `suma` almacena la suma de **todos** los elementos de la lista.
- Al terminar el `while`, la variable `i` ya **no** nos interesa.

Promedio

```
lista = [10,15,25,11,14,29,43,6,8,13]
suma = 0
i = 0
while i < len(lista):
    suma = suma + lista[i]
    i = i+1
promedio = suma/len(lista)
```

- Para sumar los elementos necesitamos **recorrer** la lista, casillero a casillero.
- Para recorrer la lista necesitamos definir una variable que recorra los **índices**.
- Una **convención** usual es llamar *i* (de *index*) a una variable que recorre índices.
- Necesitamos una variable que vaya registrando la suma **acumulando** la suma.
- Cuando termina el `while`, la variable `suma` almacena la suma de **todos** los elementos de la lista.
- Al terminar el `while`, la variable `i` ya **no** nos interesa.
- Es **red muy importante** la línea `i = i+1`. Sin eso, el programa **no termina nunca**.

Volviendo a las figus

Opción 1:

- Escribir un código que repita el llenado del álbum muchas veces y vaya rellenando una lista que almacene cuántas figus hubo que comprar cada vez.
- **Luego:** usar un código como el anterior para promediar los valores de la lista.

Volviendo a las figus

Opción 1:

- Escribir un código que repita el llenado del álbum muchas veces y vaya rellenando una lista que almacene cuántas figus hubo que comprar cada vez.
- **Luego:** usar un código como el anterior para promediar los valores de la lista.

```
import random
repeticiones = 1000
cantidades = [0]*repeticiones
paso = 0
tam_album = 6
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso +1
```

Al terminar este código, `cantidades` es una lista que guarda la cantidad de figus compradas cada vez. Luego se pueden promediar los valores de `cantidades`.

Mejorado

Opción 2:

Si queremos repetir 20 millones de veces, `cantidades` sería una lista de 20 millones de casilleros: a 8 bytes por casillero son 160 millones de bytes. O sea: 160 MB de memoria ocupados sólo para almacenar la lista.

Mejorado

Opción 2:

Si queremos repetir 20 millones de veces, `cantidades` sería una lista de 20 millones de casilleros: a 8 bytes por casillero son 160 millones de bytes. O sea: 160 MB de memoria ocupados sólo para almacenar la lista.

¿Realmente **necesitamos** generar la lista `cantidades`?

Mejorado

Opción 2:

Si queremos repetir 20 millones de veces, `cantidades` sería una lista de 20 millones de casilleros: a 8 bytes por casillero son 160 millones de bytes. O sea: 160 MB de memoria ocupados sólo para almacenar la lista.

¿Realmente **necesitamos** generar la lista `cantidades`?

Para obtener el promedio, no nos interesa el número particular de figus de cada vez, sino **sumarlos**:

```
import random
repeticiones = 1000
total = 0
paso = 0
tam_album = 6
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    total = total + cant_figus
    paso = paso + 1
print(total/repeticiones)
```

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:
 - Es más económica en recursos, porque se ahorra el almacenamiento de una lista potencialmente muy grande.

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:
 - Es más económica en recursos, porque se ahorra el almacenamiento de una lista potencialmente muy grande.
 - Implica calcular el promedio *a mano*.

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:
 - Es más económica en recursos, porque se ahorra el almacenamiento de una lista potencialmente muy grande.
 - Implica calcular el promedio *a mano*.
- Si nos interesa hacer otras cosas la lista `cantidades` puede guardar información necesaria, y en tal caso conviene usar la Opción 1.

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:
 - Es más económica en recursos, porque se ahorra el almacenamiento de una lista potencialmente muy grande.
 - Implica calcular el promedio *a mano*.
- Si nos interesa hacer otras cosas la lista `cantidades` puede guardar información necesaria, y en tal caso conviene usar la Opción 1.
 - Permite ver en qué proporción del total de las veces alcanzó con comprar 15 figuritas: o sea, qué chances tengo de llenar el álbum si compro sólo 15 figurita:s.

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:
 - Es más económica en recursos, porque se ahorra el almacenamiento de una lista potencialmente muy grande.
 - Implica calcular el promedio *a mano*.
- Si nos interesa hacer otras cosas la lista `cantidades` puede guardar información necesaria, y en tal caso conviene usar la Opción 1.
 - Permite ver en qué proporción del total de las veces alcanzó con comprar 15 figuritas: o sea, qué chances tengo de llenar el álbum si compro sólo 15 figurita:s.
 - Permite graficar todos los casos y así darnos una idea de la *variabilidad*.

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:
 - Es más económica en recursos, porque se ahorra el almacenamiento de una lista potencialmente muy grande.
 - Implica calcular el promedio *a mano*.
- Si nos interesa hacer otras cosas la lista `cantidades` puede guardar información necesaria, y en tal caso conviene usar la Opción 1.
 - Permite ver en qué proporción del total de las veces alcanzó con comprar 15 figuritas: o sea, qué chances tengo de llenar el álbum si compro sólo 15 figurita:s.
 - Permite graficar todos los casos y así darnos una idea de la *variabilidad*.
 - Permite calcular el máximo de figus que hubo que comprar para llenar el álbum, y así darnos una idea del **peor** caso y no sólo del **caso promedio**.

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:
 - Es más económica en recursos, porque se ahorra el almacenamiento de una lista potencialmente muy grande.
 - Implica calcular el promedio *a mano*.
- Si nos interesa hacer otras cosas la lista `cantidades` puede guardar información necesaria, y en tal caso conviene usar la Opción 1.
 - Permite ver en qué proporción del total de las veces alcanzó con comprar 15 figuritas: o sea, qué chances tengo de llenar el álbum si compro sólo 15 figurita:s.
 - Permite graficar todos los casos y así darnos una idea de la *variabilidad*.
 - Permite calcular el máximo de figus que hubo que comprar para llenar el álbum, y así darnos una idea del **peor** caso y no sólo del **caso promedio**.
 - Teniendo la lista, el promedio se calcula fácil con `sum(cantidades)/len(cantidades)`

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:
 - Es más económica en recursos, porque se ahorra el almacenamiento de una lista potencialmente muy grande.
 - Implica calcular el promedio *a mano*.
- Si nos interesa hacer otras cosas la lista `cantidades` puede guardar información necesaria, y en tal caso conviene usar la Opción 1.
 - Permite ver en qué proporción del total de las veces alcanzó con comprar 15 figuritas: o sea, qué chances tengo de llenar el álbum si compro sólo 15 figurita:s.
 - Permite graficar todos los casos y así darnos una idea de la *variabilidad*.
 - Permite calcular el máximo de figus que hubo que comprar para llenar el álbum, y así darnos una idea del **peor** caso y no sólo del **caso promedio**.
 - Teniendo la lista, el promedio se calcula fácil con `sum(cantidades) / len(cantidades)`

Diseño

Ambos opciones son válidas, en principio. Tomar una u otra es una elección de **diseño** de nuestro programa que depende de qué otras cosas querramos hacer.

- Si la consigna es simplemente estimar la cantidad de figuritas que es necesario comprar en promedio para llenar el álbum, la Opción 2 alcanza, teniendo en cuenta que:
 - Es más económica en recursos, porque se ahorra el almacenamiento de una lista potencialmente muy grande.
 - Implica calcular el promedio *a mano*.
- Si nos interesa hacer otras cosas la lista `cantidades` puede guardar información necesaria, y en tal caso conviene usar la Opción 1.
 - Permite ver en qué proporción del total de las veces alcanzó con comprar 15 figuritas: o sea, qué chances tengo de llenar el álbum si compro sólo 15 figurita:s.
 - Permite graficar todos los casos y así darnos una idea de la *variabilidad*.
 - Permite calcular el máximo de figus que hubo que comprar para llenar el álbum, y así darnos una idea del **peor** caso y no sólo del **caso promedio**.
 - Teniendo la lista, el promedio se calcula fácil con `sum(cantidades) / len(cantidades)`

Nos quedamos con la Opción 1

Esto recién comienza. . .

Terminamos. . . ¡sólo por hoy!